

ROS2 Middleware

Davis Onyeoguzoro

Description

ROS2 middleware is the communication layer which enables nodes to publish, subscribe, request, and respond across a distributed system using a standardized protocol.

ROS2 middleware is implemented using a DDS (Data Distribution Service) standard. This standard makes ROS2 flexible, real-time, and robust compared to ROS1.

ROS2 does not implement DDS protocol directly, it provides an abstraction layer which is the RMW (ROS Middleware Interface), this is the layer in ROS2 that abstracts the actual DDS underneath. The common RMWs are; Fast DDS, Cyclone DDS, RTI Connex.

Importance of ROS2 Middleware

1. ROS2 middleware decouples communication between ROS nodes, the nodes do not need to know each other's IP addresses or ports.
2. They handle the discovery of ROS Nodes, that way Nodes automatically find each other on the network.
3. They support Quality of Service (QoS) in other to control the reliability, delivery, deadlines and history.
4. They work over distributed networks, and enables real-time communication.

ROS2 Middleware Implementations

The DDS vendors handle the core message passing, discovery, serialization, and transport. Users can choose which DDS implementation to use by setting environment variables or configuring `rmw_implementation`.

1. **Fast DDS:** It is a complete open-source DDS implementation for real-time embedded architectures and operating systems. It is the **default middleware** used in many ROS2 distributions such as Foxy, Galactic, and Humble.

Installation

Installing Fast DDS

* Humble:

```
sudo apt install ros-humble-rmw-fastrtps-cpp
```

Switch to Fast DDS

```
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
```

Verify the current middleware

```
echo $RMW_IMPLEMENTATION
```

Pros

- Fast DDS comes pre-installed in most ROS2 distros (Humble, Foxy, etc).

- It is actively maintained and updated.

Cons

- Fast DDS consumes slightly more resources under high message throughput.
- They have slower discovery in complex networks (MQTT, and Modbus)
- It may not be optimal for strict real-time systems

2. **Cyclone DDS:** It is a high-performance open-source DDS implementation known for its low resource usage and excellent support for real-time and embedded applications.

Installation

Installing Cyclone DDS

* Humble:

```
sudo apt install ros-humble-rmw-cyclonedds-cpp
```

Switch to Cyclone DDS

```
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

Verify the current middleware

```
echo $RMW_IMPLEMENTATION
```

Pros

- Cyclone DDS has low memory and CPU footprint.
- It is fast and reliable especially on peer delivery networks.

Cons

- It is slightly less user friendly tooling and GUI support
- It may require tuning for complex QoS profiles.

3. **RTI Connex DDS:** RTI DDS is a commercial DDS that is known for industrial robustness, compliance with the full DDS specification, and powerful debugging and visualization tools.

Installation

Installing RTI Connex DDS requires license.

* Humble:

```
sudo apt update && sudo apt install -q -y rti-connex-dds-7.3.0-ros
```

Installing RTI Connex DDS binary packages

```
sudo apt update && sudo apt install -q -y ros-humble-rmw-connextdds
```

Switch to RTI Connex DDS

```
export RMW_IMPLEMENTATION=rmw_connextdds
```

Verify the current middleware

```
echo $RMW_IMPLEMENTATION
```

Pros

- RTI Connex DDS offers the best reliability and QoS features.
- It also has advanced development tools like the RTI Admin Console, Monitor).

Cons

- It requires commercial license for production use.
- It has a complex installation process.

Middleware Selection

ROS2 allows users to select from different DDS (Data Distribution Service) implementation as middleware via RMW_IMPLEMENTATION layer. the middleware selection significantly affects how ROS2 Nodes discover each other, communicate, and handle Quality of Service (QoS) policies such as reliability, durability, and deadline.

Note: If no value is set, ROS2 will use the default middleware configured during installation, which is usually Fast DDS.

To get a detailed report on your ROS2 environment, including:

- Which DDS implementation is active
- Whether your ROS environment is correctly configured
- System resource availability
- DDS-specific configuration (QoS, Discovery status)

Use the command:

```
ros2 doctor --report
```

This command outputs a system health check report. It included:

- Middleware in use (e.g Cyclone DDS, Fast DDS)
- Python and C++ client library status
- DOMAIN ID
- Multicast Discovery check
- ROS2 workspace path validation
- QoS compatibility (if nodes are already running)

Note: If multicast is not supported by the network (for instance if is blocked by the WiFi Router, discovery between ROS2 Nodes on different machines fail (this happened in the Assembly Line Project, more on this later). This is especially important when using different DDS implementations or running across multiple hosts.

```
[Sun Jun 22 13:52:44] davis@Davis-PC:/bin$ ros2 doctor --report
/opt/ros/humble/lib/python3.10/site-packages/ros2doctor/api/__init__.py: 154: UserWarning: Fail to

NETWORK CONFIGURATION
inet      : 127.0.0.1
inet4     : ['127.0.0.1']
inet6     : ['::1']
netmask   : 255.0.0.0
device    : lo
flags     : 73<LOOPBACK,RUNNING,UP>
mtu       : 65536
inet      : [REDACTED]
inet4     : [REDACTED]
ether     : [REDACTED]
inet6     : [REDACTED]
netmask   : 255.255.255.0
device    : wlp0s20f3
flags     : 4163<MULTICAST,RUNNING,UP,BROADCAST>
mtu       : 1500
broadcast : 192.168.254.255
inet      : [REDACTED]
inet4     : [REDACTED]
inet6     : [REDACTED], [REDACTED]
netmask   : 255.255.255.255
device    : tailscale0
flags     : 4305<MULTICAST,UP,PTP,RUNNING,NOARP>
mtu       : 1280
ether     : [REDACTED]
device    : enp0s31f6
flags     : 4099<MULTICAST,UP,BROADCAST>
mtu       : 1500

PLATFORM INFORMATION
system    : Linux
platform info : Linux-6.8.0-60-generic-x86_64-with-glibc2.35
release    : 6.8.0-60-generic
processor   : x86_64

QOS COMPATIBILITY LIST
compatibility status : No publisher/subscriber pairs found

RMW MIDDLEWARE
middleware name : rmw_cyclonedds_cpp

ROS 2 INFORMATION
distribution name : humble
distribution type : ros2
distribution status : active
release platforms : {'rhel': ['8'], 'ubuntu': ['jammy']}

TOPIC LIST
topic      : none
publisher count : 0
subscriber count : 0
```

Figure 1: ros2 doctor report image

Choosing ROS2 Middleware depends on application's priorities, whether it is low latency, real-time, or ease of setup.

Effects of using different DDS Implementation on a Network

ROS2 relies on a DDS-based communication layer, and each implementation uses its own implementation of the DDS specification. For instance;

1. There are differences in default configurations
2. Network discovery behaviors vary in all DDS implementations
3. Serialization and topic negotiation mismatches

How DDS Works in ROS2

When a ROS2 Node starts (Fanuc_Cart_Pose_Server):

- It creates a DDS participant.
- This participant that is created then broadcasts discovery information over multicast or unicast. All major DDS implementations support multicast for discovery by default. Multicast is when one message is sent to everyone on the local network. So the moment a Node starts, it automatically find other Nodes (publishers, subscribers, services, etc) on the same network.
- Other participants on the network try to detect each other and match topics based on names, types, and QoS settings.

Using different DDS vendors on different machines can be a problem, because;

- One DDS implementation may send discovery packets differently.
- Another may not parse the incoming packets properly.
- This breaks the chain of discovery, connection, and message exchange.

Case Study: Assembly Line Project

- Laptop A uses Fast DDS (default), and preferred for create3
- Laptop B uses Cyclone DDS

By default;

- Fast DDS uses multicast UDP for discovery
- Cyclone DDS uses a slightly different default multicast setup

Note: Multicast working in ROS2 depends entirely on the configuration of the Router or Switch. If it is absolutely necessary to use different DDS implementations; we can set a manual configuration of the DDS for interoperability.

Role of DOMAIN ID

The Domain ID is used by the DDS middleware to determine the set of UDP ports what will be used for discovery and communication. RTPS (Real-Time Publish-Subscribe) which is the DDS standard defines the formula for assigning ports.

This is important because when multiple ROS2 Nodes communicated, they must be in the same DOMAIN ID to use the same set of ports for discovery and communication

If different laptops use different ROS_DOMAIN_IDs, they will calculate different port sets and wont find each other unless a manual configuration is done.

Set Domain ID

```
export ROS_DOMAIN_ID=0
```

```
echo $ROS_DOMAIN_ID
```

Domain ID to UDP Port Calculator

Domain ID:

Participant ID:

Discovery Multicast Port:

User Multicast Port:

Discovery Unicast Port:

User Unicast Port:

Figure 2: Domain ID to UDP Port Calculator

Disabled Multicast on WiFi Router

If multicast is disabled/blocked by the WiFi Router, nodes would not be able to discover each other. There it is best to force DDS communication via unicast by specifying the peer IP address (that is the IP address of the robot or another ROS2 Node).

ROS2 via DDS uses UDP multicast by default for ROS2 Nodes discovery, this restriction can cause Nodes on the same network to fail to discover each other, even if they are otherwise connected to the same subnet and can ping each other.

As a result, ROS2 Nodes may appear to be running correctly but remain isolated, unable to exchange topics, services, or actions. This leads to sporadic behaviors, for instance during the Assembly Line Project, where `ros2 topic list` or `ros2 node list` doesn't show remote nodes, or messages where not

broadcasted on the network.

Fast DDS Unicast Configuration (XML)

The following XML configuration disables multicast and specifies a unicast peer IP for Fast DDS discovery:

```
<?xml version="1.0" encoding="UTF-8" ?>
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles">
  <participant profile_name="unicast_connection" is_default_profile="true">
    <rtps>
      <builtin>
        <metatrafficUnicastLocatorList>
          <locator>
            <udp4>
              <address>ROBOT_IP</address>
            </udp4>
          </locator>
        </metatrafficUnicastLocatorList>
        <initialPeersList>
          <locator>
            <udp4>
              <address>ROBOT_IP</address>
            </udp4>
          </locator>
        </initialPeersList>
      </builtin>
    </rtps>
  </participant>
</profiles>
```

Cyclone DDS Unicast Configuration (YAML)

The following YAML configuration disables multicast and forces DDS discovery over unicast by specifying the IP addresses of known peers. This is useful on Wi-Fi networks that block multicast.

```
CycloneDDS:
  Domain: 0 # This must be the sane across all ROS2 Nodes
  Discovery:
    multicast: false
    peers: # Static list of known ROS 2 node IP addresses
      - "172.23.255.214" # IP for Bill Fanuc
      - "172.23.255.215" # IP for DJ Fanuc
      - "172.23.255.245" # IP for Laptop A
  NetworkInterfaceAddress: auto
```

Suggestions for Stress Test

For the Stress Test, I believe it would be best if we have ROS2 (DDS), MQTT, and Modbus protocols running concurrently on the same network. This would enable us to evaluate how well the network handles multiple data streams, protocol coexistence, and potential congestion. We can continually restart ROS2 Nodes while MQTT and Modbus are running at high load. While monitoring or inspecting lost multicast packets using Wireshark or tcpdump.

We can also simulate packet loss, delay, or jitter on the network to test ROS2 QoS, and see the performance and recovery.